

面向数据中心热电耦合建模与 AI 调度的技术路线

应用场景 · 实现途径 · GitHub 开源代码 · 所需准备

研究技术路线方案

总体技术路线：先形成可解释数字环境，再建立优化基准，最后引入AI在线调度

研究主线保持“能源工程物理模型 + 算力柔性 + 智能决策”，避免直接进入黑箱算法。



最终输出：数据中心数字孪生环境 · 调度算法库 · AI控制器 · 综合评价体系 · 可扩展算电协同平台

依据：浙江大学能源工程学院算电协同工作规划、算力中心 AI 调度技术路线实现文档

版本 V1.0 2026 年 7 月

执行摘要

本报告面向 AI 算力中心的能源工程问题，围绕计算任务如何转化为 IT 功率、热负荷与制冷需求，以及如何利用电价、碳排因子、可再生能源、室外温度等外部信号实现算力与能源协同调度，形成一条从可解释物理模型到数学优化、再到 AI 在线决策的完整技术路线。

核心定位 不是研究传统云计算中的网络路由或通信寻址，而是研究“计算负荷—电力消耗—发热—温度—制冷—总功率—外部能源信号—调度决策”的闭环。

项目总体采用“一个数据底座 + 两层核心模型 + 两类调度方法 + 一个算电扩展方向”的结构。先建立单算力中心热电耦合数字环境，再构建时间迁移、热管理和多中心空间迁移的数学优化模型，以优化结果作为性能上界和专家示范数据，最后训练监督学习或强化学习调度器，实现快速在线决策。

GitHub 开源代码不作为最终成果的替代品，而作为模型、接口、数据和基线算法的可复用组件。建议以 SustainDC 学习环境组织，以 CompOpt 深化物理模型，以 Alibaba Cluster Data 提供真实 AI 负荷，以 CarbonScaler 建立时间迁移基线，以 SustainCluster 扩展多中心时空调度，后期再利用 OpenG2G 与 FlexDC-Sim 接入电网互动和需求响应。

阶段	核心任务	主要输出
阶段 0	数据底座与实验环境	统一时序数据、可复现实验配置
阶段 1	单中心热电耦合建模	任务到总功率的完整动态模型
阶段 2	单中心协同优化	时间迁移、热管理、储能可选
阶段 3	多中心时空调度	异地分配、网络/SLA/迁移成本
阶段 4	AI 在线调度	模仿学习、PPO/SAC、泛化评价
阶段 5	算电协同扩展	新能源、需求响应、配电网、VPP

目录

- 一、项目定位与研究边界
- 二、总体技术框架
- 三、主要应用场景
- 四、GitHub 开源代码利用路线
- 五、阶段 0：数据底座与实验环境准备
- 六、阶段 1：单算力中心热电耦合模型
- 七、阶段 2：单中心能源—算力优化调度
- 八、阶段 3：多算力中心时空协同调度
- 九、阶段 4：AI 智能调度模型
- 十、阶段 5：数据中心—电网协同扩展
- 十一、自研代码仓库架构
- 十二、所需准备
- 十三、实验设计与评价体系
- 十四、风险、替代方案与创新点
- 十五、最终推进顺序与验收标准
- 附录 A：GitHub 仓库清单与代码阅读顺序
- 附录 B：第一版最小可行系统（MVP）

一、项目定位与研究边界

1.1 研究对象

本项目从能源工程视角研究 AI 算力中心。研究对象包括计算任务、IT 设备、机房热环境、制冷系统、储能与可再生能源，并进一步考虑电价、区域碳排因子、室外气象、网络时延及跨区域带宽等外部条件。

计算任务 L_t → IT 功率 $P_{IT,t}$ → 发热 $Q_{IT,t}$ → 温度 T_t → 制冷功率 $P_{cool,t}$ → 总功率 $P_{DC,t}$

这一研究链条回答的不是“任务如何寻址”，而是“任务进入算力中心后对能源系统造成什么影响，以及系统如何通过调度与控制降低成本、能耗和碳排放”。

1.2 研究边界

纳入主线	暂不作为第一阶段重点
计算任务时间柔性	复杂网络路由与通信协议
IT 功率与发热	芯片级 CFD 全尺寸仿真
一阶热动态与制冷功率	水系统全部设备的精细机理
电价、碳强度、气象、绿电	选址与用水全过程评价
多中心任务迁移	真实生产集群编排平台部署
监督学习与强化学习	大模型 Agent 直接控制生产系统

研究原则 先保证物理模型可解释、优化模型可运行、对比基线可复现，再引入 AI。避免项目直接进入黑箱算法。

1.3 总体目标

- 1. 建立 AI 工作负荷、IT 功率、发热功率、机房温度、制冷功率与总用电功率之间的定量关系。
- 2. 建立考虑电价、碳排、绿电和室外温度的单中心协同优化模型。
- 3. 扩展到多个异地算力中心，研究任务在时间与空间维度的联合迁移。

- 4. 建立数学优化与 AI 调度的统一实验环境，并评价 AI 与最优解之间的差距。
- 5. 形成可扩展至储能、需求响应、配电网和虚拟电厂的自主代码平台。

二、总体技术框架



2.1 一个数据底座

所有模型共享统一时间轴和统一数据接口。最小数据向量定义为：

$$D_t = \{ L_t, \lambda_t, \rho_t, T_out,t, P_ren,t \}$$

L_t : 计算负荷; λ_t : 电价; ρ_t : 碳强度; T_out,t : 室外温度; P_ren,t : 可再生能源出力。

数据层必须独立于控制算法，确保规则策略、数学优化和 AI Agent 在完全相同的外部输入上进行对比。

2.2 两层核心模型

第一层是单算力中心内部热电耦合模型，用于描述任务、功率、热量、温度和制冷之间的动态关系。第二层是外部条件驱动的协同调度模型，用于决定任务什么时候处理、在哪里处理、是否延期或迁移。

层级	输入	主要状态/变量	输出
内部热电耦合层	计算负荷、室外温度	利用率、IT 功率、温度、制冷量	总功率、热安全状态
外部协同调度层	电价、碳强度、绿电、网络条件	任务队列、分配量、制冷动作	成本、碳排、时延、迁移量

2.3 两类调度方法

方法	作用	优点	局限
数学优化	求解给定预测条件下的近似最优策略	可解释、可作为上界	在线求解可能较慢
AI 调度	从状态快速映射到控制动作	推理快、适合高维状态	需训练并验证安全性

关键关系 数学优化不是 AI 的竞争对手，而是 AI 的性能标尺和专家教师。最推荐的路线是“优化器生成专家数据 → 模仿学习 → 强化学习微调”。

三、主要应用场景

3.1 单算力中心热电耦合运行模拟

输入某类 AI 训练或推理负荷，计算其 IT 功耗、发热、温升、制冷需求与总用电功率。用于模型验证和数字孪生底座。

3.2 电价驱动的任务时间迁移

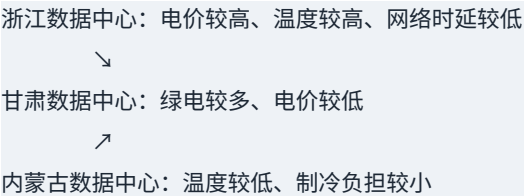
对可延迟的训练、科学计算和离线分析任务，在截止时间约束下避开高电价时段，转移至低价时段处理。

3.3 任务调度与热管理联合优化

同时决定批处理任务处理量与制冷量，利用机房热惯性进行提前制冷或延迟制冷，降低峰时功率和成本。

3.4 多算力中心时空协同调度

在不同区域的数据中心之间分配任务，综合考虑电价、碳排、绿电、气温、计算容量、网络时延和迁移成本。



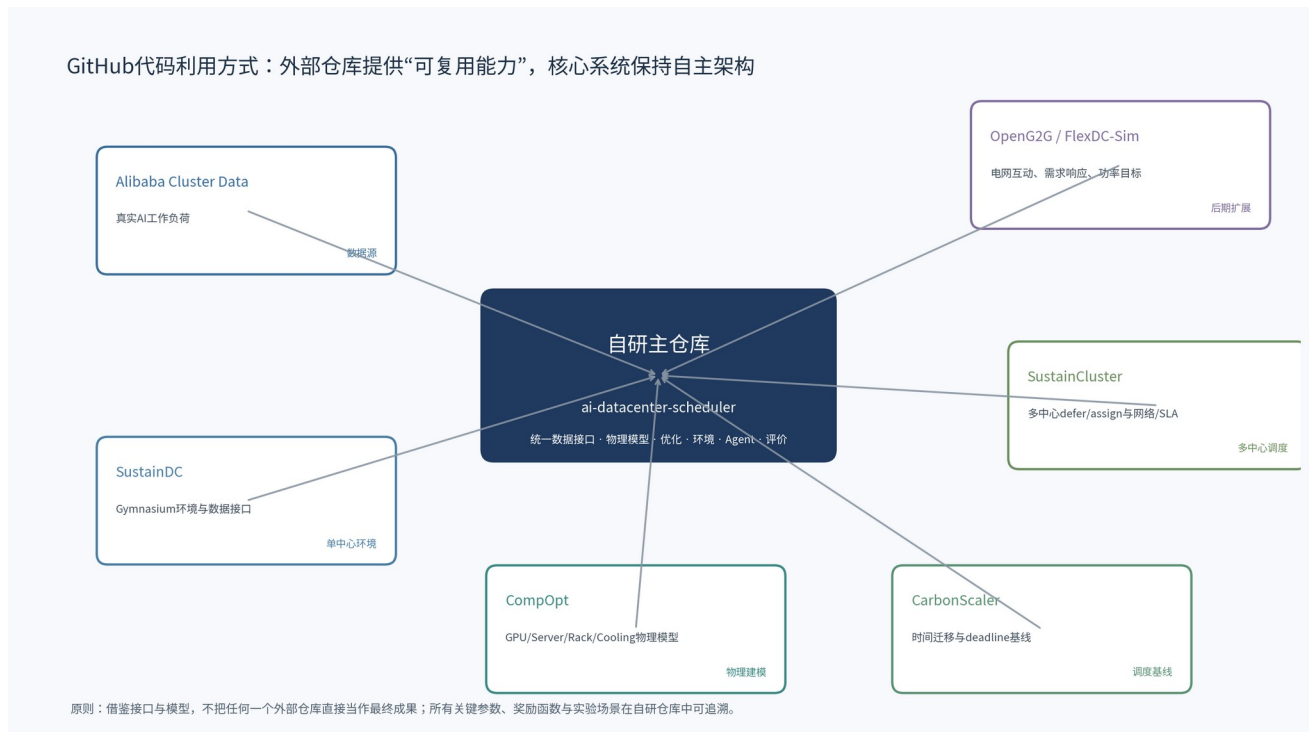
3.5 AI 在线调度

在高维状态和随机扰动下，用模仿学习、PPO 或 SAC 快速输出任务、制冷、储能或多中心分配动作。

3.6 数据中心参与需求响应与虚拟电厂

在电网下发功率目标时，通过任务延期、功率限制、制冷调节、储能和异地迁移实现功率跟踪。

四、GitHub 开源代码利用路线



4.1 总体原则

1. 不把任何一个外部仓库直接当作最终研究成果。
2. 优先复用数据接口、环境规范、模型结构和基线算法。
3. 核心物理参数、目标函数、奖励函数、实验场景和评价指标全部放入自研仓库。
4. 每个外部仓库都明确“借鉴什么”和“不照搬什么”，保证项目独立性。

仓库	项目定位	重点借鉴	暂不照搬
Alibaba / clusterdata	真实 AI 工作负荷数据	重点使用 2020 GPU Trace 的任务与资源数据；先提取一周或数天并按 15 min 聚合。	不要一开始处理超大规模全部数据。

仓库	项目定位	重点借鉴	暂不照搬
HewlettPackard / dc-rl (SustainDC)	单中心 Gymnasium 环境架构	学习主环境 reset/step、workload/weather/carbon manager、子环境交换状态。	不直接照搬完整多智能体控制和原奖励函数。
HewlettPackard / compopt	AI 数据中心物理模型	学习 GPU Chip—Server—Rack—CDU—Cooling Tower 的层级建模和热 RC 网络。	第一版不直接采用全部高复杂度模型。
umassos / CarbonScaler	任务时间迁移基线	借鉴 deadline-aware、低碳时段增加计算的调度逻辑。	当前不复现 Kubernetes 部署。
HewlettPackard / sustain-cluster	多中心时空调度	借鉴 defer/assign 动作、网络成本、SLA、价格和碳强度输入。	不直接把仓库结果作为自己的最终创新。
Green Software Foundation / carbon-aware-sdk	碳强度接口	后续标准化获取时间/地域碳信号。	第一阶段不必部署完整服务。
gpu2grid / openg2g	数据中心—电网互动	后期研究数据中心功率变化对配电网的影响。	不放入当前最小可行版本。
peaclab / flexdc-sim	需求响应与功率目标	后期研究电网功率目标、任务与节点功率管理。	不作为当前主线底座。

4.2 推荐下载顺序

1. `git clone https://github.com/HewlettPackard/dc-rl.git sustaindc`
2. `git clone https://github.com/HewlettPackard/compopt.git`
3. `git clone https://github.com/alibaba/clusterdata.git`
4. `git clone https://github.com/umassos/CarbonScaler.git`
5. `git clone https://github.com/HewlettPackard/sustain-cluster.git`

第一轮只要求前三个仓库能够阅读和局部运行。CarbonScaler 与 SustainCluster 在单中心模型跑通后再进入。

五、阶段 0：数据底座与实验环境准备

5.1 目标

建立统一的数据目录、时间步长、配置文件和实验记录规范。所有后续模型均从同一数据接口读取输入，避免不同算法使用不同数据导致结果不可比。

5.2 AI 工作负荷数据

建议使用 Alibaba Cluster Data 作为真实工作负荷来源。第一版优先使用 2020 GPU Trace，提取任务开始/结束时间、GPU/CPU 需求和任务类别等信息。

原始 Job / Task 数据
↓
筛选 GPU 任务
↓
提取开始时间、结束时间、资源请求量
↓
按 15 min 聚合
↓
分离 online_load 与 batch_load
↓
归一化得到 L_t

字段	第一版用途	处理方式
timestamp	统一时间轴	转换到固定 15 min 步长
GPU/CPU request	构造计算负荷	加权并归一化
start / end time	计算并发任务量	按时间窗聚合
task type (若可用)	区分在线与批处理	没有标签时可规则划分

5.3 外部信号数据

数据	第一版来源	后续升级
室外温度	典型日或历史天气 CSV	多地区真实气象
电价	人工构造峰平谷电价	真实市场或区域电价

数据	第一版来源	后续升级
碳强度	历史时间序列或人工场景	Carbon Aware SDK
光伏/风电	人工生成典型出力	真实新能源历史数据
网络时延	固定矩阵	真实区域时延与带宽

5.4 建议统一数据格式

```
timestamp, online_load, batch_load, price, carbon, outdoor_temp, renewable
2026-01-01 00:00, 0.42, 0.33, 0.35, 510, 4.2, 0.00
2026-01-01 00:15, 0.45, 0.36, 0.35, 505, 4.0, 0.00
```

验收标准 任意一天的数据可以通过一个脚本读取，并同时画出工作负荷、电价、碳强度、温度和可再生能源曲线。

六、阶段 1：单算力中心热电耦合模型

6.1 应用场景与输入输出

输入	中间状态	输出
计算负荷 L_t	利用率 u_t 、IT 功率、热负荷	总功率 $P_{DC,t}$
室外温度 $T_{out,t}$	机房温度 T_t 、制冷量	温度安全状态

6.2 计算负荷建模

$$L_t = L_{on,t} + L_{bat,t}$$

交互式负荷需要实时处理，批处理负荷允许在给定窗口内延迟。第一版可采用 70% 实时任务、30% 批处理任务作为可调参数，后续再根据真实 trace 校准。

6.3 IT 功率模型

$$u_t = L_t / C_{max}$$

$$P_{IT,t} = P_{idle} + (P_{peak} - P_{idle}) \cdot u_t$$

第一版使用线性利用率模型，便于参数解释和优化求解。第二版可拆分 GPU、CPU、内存和网络功率。

6.4 发热模型

$$Q_{IT,t} \approx P_{IT,t}$$

对于数据中心能量平衡，IT 设备输入电能最终大部分以热量形式释放。该近似用于连接计算负荷与热管理。

6.5 机房热动态模型

$$T_{t+1} = T_t + \Delta t / C_{th} \cdot [Q_{IT,t} - Q_{cool,t} - (T_t - T_{out,t}) / R_{th}]$$

$$T_{min} \leq T_t \leq T_{max}$$

该模型能够体现热惯性：制冷不必每时刻完全跟随发热功率，而可以在安全温度范围内提前或延后。

6.6 制冷功率模型

$$P_{cool,t} = Q_{cool,t} / COP_t$$

第一版令 COP 为常数。第二版再建立 COP 与室外温度的关系，例如 $COP_t = a - b \cdot T_{out,t}$ ，或使用经验曲线。

6.7 数据中心总功率

$$P_{DC,t} = P_{IT,t} + P_{cool,t} + P_{aux,t}$$

若加入储能和新能源，则进一步计算电网购电功率：

$$P_{grid,t} = P_{DC,t} + P_{ch,t} - P_{dis,t} - P_{ren,use,t}$$

6.8 SustainDC 与 CompOpt 的具体作用

项目	当前阶段重点看什么	转化为自研模块
SustainDC	主环境 reset/step、数据管理器、子环境状态交换	single_dc_env.py、data_manager.py
CompOpt	热 RC 网络、芯片到机架的层次、JointDC 环境	thermal.py、cooling.py 的升级路径

验收标准 输入一条 24 h 计算负荷与室外温度曲线，能够稳定输出 IT 功率、发热、机房温度、制冷功率和总功率，并保证能量方向与物理趋势合理。

七、阶段 2：单中心能源—算力优化调度

7.1 核心问题

在满足计算容量、任务截止时间和温度安全约束的条件下，决定批处理任务什么时候执行，以及制冷系统什么时候提供多少冷量。

7.2 任务队列

$$B_{(t+1)} = B_t + L_{bat,t} - x_{bat,t}$$

B_t 表示尚未完成的任务， $x_{bat,t}$ 表示本时段实际处理量。该状态方程体现任务的时间转移能力。

7.3 决策变量

第一版	第二版扩展
批处理量 $x_{bat,t}$	储能充放电 $P_{ch,t}$ 、 $P_{dis,t}$
制冷量 $Q_{cool,t}$	SOC _t

7.4 目标函数

$$\min \sum_t [\lambda_t P_{grid,t} \Delta t + w_c p_t P_{grid,t} \Delta t + w_d B_t + w_T C_{thermal,t}]$$

目标函数同时考虑购电成本、碳排、任务延迟和热安全风险。权重需要通过场景分析和敏感性分析确定。

7.5 CarbonScaler 的使用方式

CarbonScaler 主要作为“可延迟任务的时间迁移基线”。当前项目只借鉴其 deadline-aware 逻辑：在低成本或低碳时段增加计算，在高成本或高碳时段减少计算，同时保证任务截止时间。

任务到达 → 截止时间 → 未来电价/碳强度 → 决定各时段处理量

7.6 推荐实验场景

场景	时间迁移	热管理	储能	用途
S0 无控制	否	被动	否	基准
S1 时间调度	是	被动	否	验证算力柔性
S2 热电联合	是	优化	否	验证热惯性价值
S3 储能扩展	是	优化	是	源荷储协同

阶段关键输出 除优化结果外，还要保存每个时刻的状态 s_t 与最优动作 a^*_t ，形成后续模仿学习的专家数据集。

八、阶段 3：多算力中心时空协同调度

8.1 建模对象

$$d = 1, 2, \dots, D$$

每个数据中心具有不同的电价、碳强度、室外温度、可再生能源、最大计算容量和网络条件。

$$\{ \lambda_{d,t}, \rho_{d,t}, T_{out,d,t}, P_{ren,d,t}, C_{max,d} \}$$

8.2 任务分配变量

$$y_{d,t} = \text{时刻 } t \text{ 分配给数据中心 } d \text{ 的任务量}$$

$$\sum_d y_{d,t} = L_t$$

8.3 成本与约束

- 计算容量约束： $y_{d,t}$ 不超过中心剩余能力。
- 网络时延约束：交互式任务必须满足 SLA。
- 迁移成本：跨区域传输和数据搬迁产生额外成本。
- 热安全约束：每个中心都必须满足温度上限。
- 任务守恒：所有任务最终必须被处理或在允许窗口内延期。

$$C_{total} = C_{ele} + C_{carbon} + C_{mig} + C_{network} + C_{delay}$$

8.4 SustainCluster 的具体利用

SustainCluster 与本项目最接近的部分是 defer/assign 动作。对于每个任务，动作可以表示“延期”或“分配到某个数据中心”。建议重点研究其动作编码、SLA 处理、网络成本和多中心状态构造。

```

action = 0    → 延期任务
action = 1    → 分配到 DC1
action = 2    → 分配到 DC2
...
action = D    → 分配到 DCD

```

8.5 推进方式

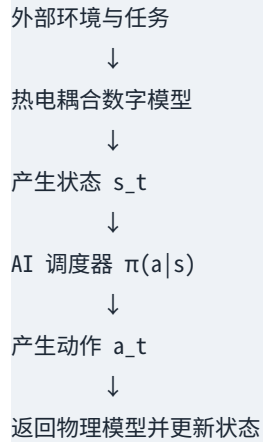
1. 先建立两个虚拟数据中心，只设置电价和温度差异。
2. 扩展到三个中心，形成“低价高碳、低碳高价、低温高绿电”三类典型中心。
3. 加入网络时延和迁移成本。
4. 最后允许任务同时进行时间迁移和空间迁移，形成真正的时空联合调度。

基线策略	说明
Random	随机分配
Round Robin	轮询分配
Most Available	分配给剩余容量最多的中心
Lowest Price	最低电价优先
Lowest Carbon	最低碳强度优先
Optimization	数学优化上界
AI	在线策略

九、阶段 4：AI 智能调度模型

9.1 基本原则

AI 的位置 AI 是控制器，物理模型是环境。AI 不直接替代热电耦合模型，而是在每个时刻读取状态、输出动作，再由物理模型计算下一状态。



9.2 路线一：监督学习 / 模仿学习

训练数据： $\{ (s_t, a^*_t) \}$ ，其中 a^*_t 来自数学优化器

$$\pi_{\theta}(s_t) \approx a^*_t$$

这是最推荐的第一个 AI 实验。它实现难度低、目标明确，并能直接评价 AI 对优化器的近似能力。

9.3 路线二：强化学习

单中心状态可定义为：

$$s_t = [L_t, B_t, \lambda_t, \rho_t, T_{out,t}, T_t, P_{ren,t}, SOC_t]$$

单中心动作可定义为：

$$a_t = [x_{bat,t}, Q_{cool,t}]$$

多中心时加入任务分配变量 $y_{d,t}$ 。奖励函数采用负成本形式：

$$r_t = - (C_{ele,t} + w1 C_{carbon,t} + w2 C_{delay,t} + w3 C_{thermal,t} + w4 C_{migration,t})$$

9.4 算法顺序

规则策略 → 模仿学习 → PPO → SAC → 多智能体算法（可选）

动作离散时可以考虑 DQN；连续控制优先使用 PPO 或 SAC。第一版推荐 PPO，因为训练流程清晰、工程实现成熟。

9.5 GitHub 代码对应

仓库	重点代码/能力	项目中的用途
SustainDC	Gymnasium、多子环境、RL 基线	单中心 AI 环境
CompOpt	JointDC、PPO 示例、物理控制基线	热管理 RL 参考
SustainCluster	SAC/PPO、评估 Notebook、rewards/configs	多中心 AI 调度

9.6 AI 评价指标

$$\text{Gap} = (J_{\text{AI}} - J_{\text{OPT}}) / J_{\text{OPT}}$$

指标类别	具体指标
最优性	AI 与优化器目标值差距
实时性	平均单次决策时间、P95 决策时间
泛化性	更换电价、气温、负荷、中心容量后的性能
安全性	温度越界、SLA 违约、容量越界
稳定性	多随机种子均值与方差

十、阶段 5：数据中心—电网协同扩展

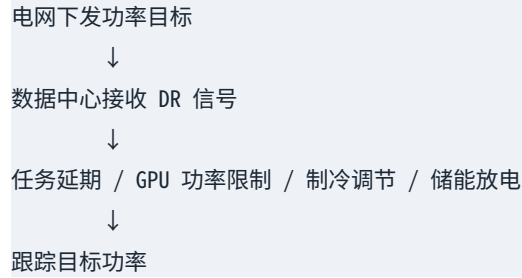
该阶段必须在前述数字环境、优化与 AI 调度基本完成后再进入。当前主线不建议同时开展配电网精细潮流。

10.1 OpenG2G：数据中心到配电网

AI 任务 → P_DC → 配电网潮流 → 节点电压 V_i

用于研究 AI 负荷变化对配电网运行的影响，以及数据中心如何根据电网状态调整功率。

10.2 FlexDC-Sim：需求响应与功率目标



10.3 可选扩展方向

- 数据中心参与虚拟电厂与辅助服务。
- 风光储算一体化运行。
- 节点电价与配电网潮流。
- Physics AI：使用 PINN、神经算子或降阶模型替代部分高成本仿真。
- 多智能体：工作负荷、制冷、储能分别作为 Agent。

十一、自研代码仓库架构

```
ai-datacenter-scheduler/  
|  
|—— configs/  
|   |—— datacenter.yaml  
|   |—— experiment.yaml  
|   |—— optimization.yaml  
|  
|—— data/  
|   |—— workload/  
|   |—— weather/  
|   |—— electricity_price/  
|   |—— carbon/  
|   |—— renewable/  
|  
|—— models/  
|   |—— workload.py  
|   |—— it_power.py  
|   |—— thermal.py  
|   |—— cooling.py
```

```
|   |—— battery.py
|   |—— datacenter.py
|
|—— optimization/
|   |—— baseline.py
|   |—— temporal_shift.py
|   |—— thermal_optimization.py
|   |—— storage_optimization.py
|   |—— multi_dc_optimization.py
|
|—— envs/
|   |—— single_dc_env.py
|   |—— multi_dc_env.py
|
|—— agents/
|   |—— imitation_agent.py
|   |—— ppo_agent.py
|   |—— sac_agent.py
|
|—— experiments/
|   |—— exp01_no_control.py
|   |—— exp02_time_shift.py
|   |—— exp03_thermal_control.py
|   |—— exp04_multi_dc.py
|   |—— exp05_ai.py
|
|—— evaluation/
|   |—— energy_metrics.py
|   |—— carbon_metrics.py
|   |—— thermal_metrics.py
|   |—— sla_metrics.py
|
|—— results/
```

11.1 代码工程要求

- 所有参数写入 YAML/JSON 配置，不硬编码在算法中。
- 每次实验记录 Git commit、随机种子、配置文件和输出目录。
- 物理模型与控制器解耦，同一环境可以接规则、优化和 AI。
- 所有评价指标集中在 evaluation/，避免不同实验重复计算。
- 外部仓库只作为依赖或参考，不在其源码中直接长期开发。

十二、所需准备

12.1 软件环境

类别	建议
操作系统	macOS / Linux 均可；复杂训练建议 Linux
Python	3.10 或 3.11
数据分析	NumPy、Pandas、Matplotlib
优化	CVXPY 或 Pyomo；求解器可先用 HiGHS/SCIP，条件允许再用 Gurobi
强化学习	Gymnasium、Stable-Baselines3；多智能体可后续引入 RLlib
配置与实验	PyYAML、TensorBoard、Weights & Biases（可选）
版本管理	Git + GitHub

12.2 编程基础

- Python 面向对象编程
- NumPy/Pandas 时间序列处理
- Matplotlib 结果可视化
- Git 分支、commit、README 和 issue 基础

12.3 数学基础

方向	最低要求
优化	线性规划、混合整数规划、约束、多目标权衡
动态系统	状态方程、离散时间系统、一阶 RC 模型
AI	神经网络、监督学习、MDP、PPO/SAC 基础

12.4 最低限度物理参数

参数	含义	第一版来源
C_max	最大计算能力	归一化设定或 trace 标定
P_idle	空载 IT 功率	文献/设备参数/开源默认值
P_peak	满载 IT 功率	文献/设备参数/开源默认值
R_th	等效热阻	文献或标定
C_th	等效热容	文献或标定
COP	制冷性能系数	典型值或经验曲线
Q_cool,max	最大制冷能力	容量假设
T_min/T_max	温度安全边界	工程规范或文献

参数使用原则 第一版允许使用文献参数、GitHub 默认参数和合理假设，但必须进行参数敏感性分析，并明确哪些结论依赖参数、哪些结论具有稳健性。

十三、实验设计与评价体系

13.1 最终实验矩阵

场景	任务时间迁移	热管理	空间迁移	AI
S0 无控制	×	×	×	×
S1 时间调度	✓	×	×	×
S2 热电联合	✓	✓	×	×
S3 多中心	✓	✓	✓	×

场景	任务时间迁移	热管理	空间迁移	AI
S4 AI 调度	✓	✓	✓	✓

13.2 评价指标

维度	指标
能耗	总用电量、IT 用电量、制冷用电量、峰值负荷、峰谷差、PUE
经济性	总购电成本、单位任务成本、峰时成本占比、迁移成本
低碳	总碳排、单位任务碳排、绿电消纳率、高碳时段负荷削减量
热安全	最高温度、越界次数、接近上限持续时间、最大制冷负荷率
服务质量	平均延迟、批处理完成率、SLA 满足率、异地迁移比例
AI	最优性差距、推理时间、泛化性能、多随机种子稳定性

13.3 必须做的对照实验

1. 同一数据输入下，对比无控制、规则、数学优化和 AI。
2. 改变电价权重与碳排权重，分析策略如何变化。
3. 改变 R_{th} 、 C_{th} 、COP，验证热模型参数敏感性。
4. 在训练分布之外的负荷和气温下测试 AI 泛化。
5. 记录数学优化与 AI 的单次决策时间，证明在线调度价值。

十四、风险、替代方案与创新点

14.1 主要风险与替代方案

风险	表现	替代方案
真实数据过大	数据清洗时间长	先截取一周并聚合到 15 min
热模型参数缺失	难以标定 R_{th}/C_{th}	文献参数 + 敏感性分析

风险	表现	替代方案
优化问题过复杂	求解时间过长	先连续化/减少中心数/缩短时域
RL 训练不稳定	奖励震荡、约束违例	先模仿学习，再 RL 微调；增加安全层
创新性被开源项目覆盖	与 SustainCluster 过于相似	突出能源工程物理模型与“优化→AI”递进体系
范围过大	同时做电网、储能、VPP	锁定单中心→多中心→AI 三阶段主线

14.2 可形成的创新点

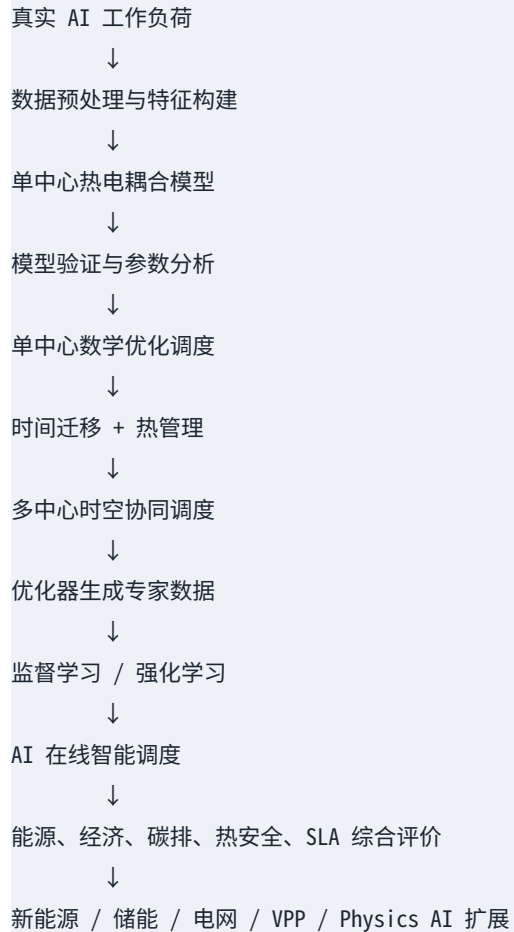
- 1. 建立面向 AI 工作负荷的可解释热电耦合数字环境，而不是只做抽象服务器任务调度。
- 2. 将任务时间迁移与热惯性联合建模，研究“算力柔性 + 热管理”的协同价值。
- 3. 建立数学优化、专家示范、模仿学习和强化学习之间的统一递进路线。
- 4. 研究多中心电价、碳排、气温和网络条件共同驱动的时空调度。
- 5. 形成可扩展到需求响应、配电网和 Physics AI 的自主软件框架。

十五、最终推进顺序与验收标准

阶段	必须完成	验收标准
0 数据准备	统一 CSV/Parquet 与配置	一键画出全部外部曲线
1 物理模型	L→P_IT→Q_IT→T→P_cool→P_DC	24h 仿真稳定，趋势合理
2 单中心优化	时间迁移 + 热管理	相对无控制有可解释收益
3 多中心	时空迁移 + 网络/SLA	不同外部条件下策略可解释
4 AI	模仿学习 + PPO/SAC	给出与优化器差距和推理速度
5 扩展	选择一个方向	形成独立扩展实验

15.1 最终技术路线概括

一句话概括 以真实 AI 工作负荷为输入，以数据中心热电耦合模型为物理基础，以数学优化为性能基准，以监督学习和强化学习为在线决策手段，研究计算任务在时间、空间和能源维度上的协同调度。



附录 A: GitHub 仓库清单与代码阅读顺序

以下链接均为本技术路线建议优先研究的公开仓库。建议先阅读 README 和示例，再进入核心环境与模型文件。

A.1 Alibaba / clusterdata

仓库地址: <https://github.com/alibaba/clusterdata>

项目定位: 真实 AI 工作负荷数据

优先研究: 重点使用 2020 GPU Trace 的任务与资源数据; 先提取一周或数天并按 15 min 聚合。

当前不建议: 不要一开始处理超大规模全部数据。

A.2 HewlettPackard / dc-rl (SustainDC)

仓库地址: <https://github.com/HewlettPackard/dc-rl>

项目定位: 单中心 Gymnasium 环境架构

优先研究: 学习主环境 reset/step、workload/weather/carbon manager、子环境交换状态。

当前不建议: 不直接照搬完整多智能体控制和原奖励函数。

A.3 HewlettPackard / compopt

仓库地址: <https://github.com/HewlettPackard/compopt>

项目定位: AI 数据中心物理模型

优先研究: 学习 GPU Chip—Server—Rack—CDU—Cooling Tower 的层级建模和热 RC 网络。

当前不建议: 第一版不直接采用全部高复杂度模型。

A.4 umassos / CarbonScaler

仓库地址: <https://github.com/umassos/CarbonScaler>

项目定位: 任务时间迁移基线

优先研究: 借鉴 deadline-aware、低碳时段增加计算的调度逻辑。

当前不建议: 当前不复现 Kubernetes 部署。

A.5 HewlettPackard / sustain-cluster

仓库地址: <https://github.com/HewlettPackard/sustain-cluster>

项目定位：多中心时空调度

优先研究：借鉴 defer/assign 动作、网络成本、SLA、价格和碳强度输入。

当前不建议：不直接把仓库结果作为自己的最终创新。

A.6 Green Software Foundation / carbon-aware-sdk

仓库地址：<https://github.com/Green-Software-Foundation/carbon-aware-sdk>

项目定位：碳强度接口

优先研究：后续标准化获取时间/地域碳信号。

当前不建议：第一阶段不必部署完整服务。

A.7 gpu2grid / openg2g

仓库地址：<https://github.com/gpu2grid/openg2g>

项目定位：数据中心—电网互动

优先研究：后期研究数据中心功率变化对配电网的影响。

当前不建议：不放入当前最小可行版本。

A.8 peaclab / flexdc-sim

仓库地址：<https://github.com/peaclab/flexdc-sim>

项目定位：需求响应与功率目标

优先研究：后期研究电网功率目标、任务与节点功率管理。

当前不建议：不作为当前主线底座。

A.9 推荐代码阅读顺序

1. SustainDC：先看 README → 主环境入口 → data manager → workload/data center 子环境。
2. CompOpt：先跑最小示例 → 看环境注册 → 看 DataCenterModel → 再看热 RC 组件。
3. Alibaba Cluster Data：先看 2020 GPU Trace README 与 Notebook → 只抽取小时间窗。
4. CarbonScaler：只看调度思想与实验配置，不先进入 Kubernetes 细节。
5. SustainCluster：先看动作空间和 observation → 再看 rewards → 最后看训练脚本。

附录 B：第一版最小可行系统（MVP）

第一版系统只需要完成一个单算力中心、24 h、15 min 时间步长的仿真和优化，不需要多中心、不需要储能、不需要电网。

模块	最低功能
data_loader.py	读取负荷、气温、电价、碳强度
it_power.py	利用率到 IT 功率
thermal.py	一阶 RC 温度更新
cooling.py	冷量到制冷功率
datacenter.py	计算总功率
baseline.py	任务到达即处理
optimizer.py	优化批处理量与制冷量
plot_results.py	输出六类核心曲线与对比指标

B.1 第一版输入

- 96 个 15 min 时间点
- 一条计算负荷曲线
- 一条室外温度曲线
- 一条电价曲线
- 一条碳强度曲线

B.2 第一版输出

- IT 功率曲线
- 机房温度曲线
- 制冷功率曲线
- 总功率曲线

- 任务积压曲线
- 优化前后成本与碳排对比

完成 MVP 后再扩展 只有当“输入一条负荷 → 得到完整能量与温度结果 → 优化策略明显改变结果”这一闭环跑通后，才进入多中心和 AI。

参考资料与来源说明

本报告基于用户提供的《浙江大学能源工程学院算电协同工作规划》和《算力中心 AI 调度技术路线实现文档》整合，并结合下列开源项目形成实施路线。GitHub 链接用于代码学习、数据获取和基线复现。

Alibaba / clusterdata: <https://github.com/alibaba/clusterdata>

HewlettPackard / dc-rl (SustainDC) : <https://github.com/HewlettPackard/dc-rl>

HewlettPackard / compopt: <https://github.com/HewlettPackard/compopt>

umassos / CarbonScaler: <https://github.com/umassos/CarbonScaler>

HewlettPackard / sustain-cluster: <https://github.com/HewlettPackard/sustain-cluster>

Green Software Foundation / carbon-aware-sdk: <https://github.com/Green-Software-Foundation/carbon-aware-sdk>

gpu2grid / openg2g: <https://github.com/gpu2grid/openg2g>

peaclab / flexdc-sim: <https://github.com/peaclab/flexdc-sim>